PsyClick Secure

Security Testing Report

CS0029 — Information Assurance and Security — Standalone Deep-Dive Deliverable

Field	Value
System	PsyClick Secure (psyclick-secure build)
Test date	10–11 July 2026
Test environment	Linux sandbox (source-level SAST, SCA, and unit/regression testing); target production platform is Windows 10/11 (required for the DPAPI-dependent backup test, SEC-05)
Tooling	bandit 1.9.4 (SAST), npm audit (SCA), pytest 9.1.1 (regression suite), manual source review (dynamic/penetration test plan)
Out of scope / not executed	Live authorized penetration test against a running deployment, dynamic DAST scan over HTTP, clean target-workstation installation test — all recorded as a test plan (Section 6), consistent with the CS0029 specification's allowance for “Simulation or Conceptual” penetration testing

1. Purpose and Scope

This report is the standalone, in-depth Security Testing Report referenced from Chapter 5 (Sections 5.4.1–5.4.5) of the PsyClick thesis and required as a distinct CS0029 Section IX deliverable. It documents every test actually executed against the psyclick-secure source tree between 10–11 July 2026, presents the complete, unfiltered findings from each tool, and separates genuine automated evidence from the conceptual/simulated penetration-test walkthrough the specification explicitly permits in place of a live authorized engagement.

No result in this report is fabricated or extrapolated. Where a test was not executed (e.g., dynamic HTTP penetration testing against a live deployment), that is stated explicitly rather than presented as a pass.

2. Test Methodology

- Static Application Security Testing (SAST) — bandit, run against all ten first-party backend Python modules (4,094 lines of code).
- Software Composition Analysis (SCA) / dependency audit — npm audit against the Electron/React frontend's full dependency tree, before and after applying non-breaking remediation.
- Security regression test suite — pytest, ten automated tests covering authentication, session management, consent, audit integrity, encrypted backup/restore, and the new intrusion monitor.
- Conceptual/simulated penetration test — a structured manual walkthrough of ten attacker scenarios against the reviewed source code, each mapped to an expected control and a pass/fail-by-design judgment, in lieu of a live authorized network engagement.

3. Static Application Security Testing (SAST) Results

Tool: bandit v1.9.4. Command: bandit <10 modules> -f json. Result: 0 High-severity findings, 13 Medium-severity findings (all rule B608), 15 Low-severity findings (all rule B110), across 4,094 scanned lines.

3.1 Medium-severity findings (B608 — possible SQL injection via string-based query construction)

File	Line(s)	Rule	Severity	Confidence	Description
api_server.py	278–286, 299, 324, 349, 369, 644	B608	Medium	Medium	Possible SQL injection vector through string-based query construction (dashboard counts, session listing/lookup, deletion)
database_manager.py	525	B608	Medium	Low	Possible SQL injection vector through string-based query construction (parameterized INSERT column-list assembly)

Manual review disposition: in every flagged line, the f-string interpolates a static, internally-chosen query fragment (a bind-placeholder character for cross-database compatibility, or a fixed WHERE-clause suffix chosen from a small internal set) — never a caller-supplied value. Every value that originates from a request (patient/session/clinician identifiers) is passed through parameterized `?` bind parameters in the same `execute()` call, which is the actual SQL-injection defense. All 13 findings are assessed as false positives of bandit's B608 heuristic, which pattern-matches on f-string SQL assembly regardless of whether the interpolated fragment is attacker-controlled. Recommended hardening (tracked in the Risk Assessment Report, R-01): refactor the dynamic-filter helper functions to avoid f-string SQL assembly entirely (e.g., a small whitelist-driven query builder), removing the need for manual re-review on every future change.

3.2 Low-severity findings (B110 — try/except/pass)

File	Line(s)	Rule	Severity	Confidence	Description
api_server.py	536	B110	Low	High	Broad try/except/pass around an optional report-export step
backend_controller.py	418	B110	Low	High	Broad try/except/pass around an optional telemetry cleanup step
database_manager.py	151, 355, 378	B110	Low	High	Broad try/except/pass around optional migration/compat paths
dynamics_logger.py	92, 105, 117, 143, 169, 178, 194	B110	Low	High	Broad try/except/pass around best-effort local telemetry logging
intrusion_monitor.py	103	B110	Low	High	Broad try/except/pass around the optional audit-log callback (prevents a logging failure from blocking a security decision)
supabase_sync.py	210, 316	B110	Low	High	Broad try/except/pass around optional cloud-sync steps (fail silently, not fail open on security-relevant paths)

Disposition: these are broad exception handlers around optional, non-security-critical paths (best-effort telemetry logging, optional cloud sync, optional report export cleanup). None guard an authentication, authorization, or consent decision — those paths use specific exception handling and fail closed (see Risk Register R-07, R-04). Recommended

follow-up: replace silent `pass` with at least a debug-level log line so operational failures in these paths are not invisible to an administrator.

4. Dependency / Software Composition Analysis (SCA) Results

Tool: npm audit, against psyclick-secure/frontend/package-lock.json. Initial scan: 16 advisories (1 critical, 9 high, 5 moderate, 1 low).

4.1 Findings before remediation

Package	Severity	Advisory summary	Dependency type
@babel/core	Moderate	Arbitrary file read via sourceMappingURL comment	devDependency (build tool)
electron	High	17 advisories: ASAR bypass, AppleScript injection, IPC spoofing, UAF issues, etc.	devDependency (packaging runtime)
esbuild / vite	Moderate	Dev server accepts requests from any website	devDependency (dev server only)
form-data	High	CRLF injection via unescaped multipart fields	transitive devDependency
joi	Moderate	Uncaught RangeError on deeply nested input	transitive devDependency
js-yaml	Moderate	Quadratic-complexity DoS in merge-key handling	transitive devDependency
react-router / react-router-dom	High	Open redirect via protocol-relative URL reinterpretation	runtime dependency
shell-quote	Critical	quote() does not escape newlines in object .op values	transitive devDependency
tar	High	Hardlink/symlink path traversal (multiple CVEs)	transitive devDependency (electron-builder)
tmp	High	Path traversal via unsanitized prefix/postfix	transitive devDependency

4.2 Remediation applied

`npm audit fix` (non-breaking changes only) was executed on 11 July 2026. Result: 16 → 8 advisories. Resolved advisories include the critical shell-quote issue and the react-router-dom open-redirect issue in a shipped runtime dependency. Remaining 8 advisories (7 high, 1 moderate) are entirely contained within build-time devDependencies — Electron itself, the esbuild/Vite development server, and the tar/electron-builder packaging toolchain — none of which ship inside the distributed application bundle presented to a clinician end user.

Zero advisories remain in the shipped runtime dependency set (react, react-dom, react-router-dom, recharts, lucide-react, clsx, date-fns) after remediation. Clearing the remaining 8 requires a breaking major-version upgrade (`npm audit fix --force`, e.g. Electron 30→43, Vite 5→8) and is scheduled as a dedicated maintenance task (Risk Assessment Report, R-14) rather than applied automatically during this review to avoid destabilizing the packaged build without a full regression pass.

5. Security Regression Test Suite Results

Command: pytest tests/test_security_controls.py -v. Result: 9 passed, 1 failed, in 1.14s (10 tests total).

Test ID	Control under test	Automated test(s)	Result (this environment)	Notes
SEC-01	Password hashing and policy	SecurityControlTests.test_password_is_hashed_and_first_account_is_admin + test_password_policy_rejects_weak_password	PASS	PASS
SEC-02	Session revocation	SecurityControlTests.test_security_session_expires_on_revocation	PASS	PASS
SEC-03	Consent fail-closed	SecurityControlTests.test_consent_is_persisted_and_fail_closed	PASS	PASS
SEC-04	Audit chain tamper detection	SecurityControlTests.test_audit_chain_detects_tampering	PASS	PASS
SEC-05	Encrypted backup + restore validation	SecurityControlTests.test_encrypted_backup_passes_restore_validation	FAIL (platform)	Expected PASS on Windows
SEC-12a	No throttle below threshold	IntrusionMonitorTests.test_no_throttle_below_threshold	PASS	PASS
SEC-12b	Throttle triggers + security audit alert	IntrusionMonitorTests.test_throttle_triggers_after_threshold_and_logs_security_audit	PASS	PASS
SEC-12c	Success clears failure history	IntrusionMonitorTests.test_successful_login_clears_failure_history	PASS	PASS
SEC-12d	Rotating-source attack on one account detected	IntrusionMonitorTests.test_rotating_source_against_single_target_still_detected	PASS	PASS

The single failure (SEC-05) is a platform limitation, not a code defect: security_manager.py's encryption calls into ctypes.windll (Windows DPAPI), which does not exist on the Linux sandbox used to run this test suite. The test asserts, and the code correctly enforces, that protected exports/backups require Windows DPAPI and raises a clear RuntimeError rather than silently falling back to plaintext when DPAPI is unavailable — which is itself a secure failure mode. This test is expected to pass on the Windows target deployment platform; independent confirmation on a Windows workstation remains an external evidence item (see Section 8).

6. Conceptual / Simulated Penetration Test

Per CS0029 Section V, penetration testing may be "Simulation or Conceptual." No live authorized engagement against a running deployment was performed. The following ten scenarios were instead walked through analytically against the reviewed source code, each with a concrete precondition, method, and code-grounded expected outcome.

#	Scenario	Method	Expected outcome (by design)	Grounding
1	Password guessing	Repeated login attempts against one account	Locked after 5 failures for 15 min; further attempts from that source throttled after 8 failures/5 min	database_manager.py MAX_FAILED_ATTEMPTS/LOCKOUT_MINUTES; intrusion_monitor.py
2	Session reuse after logout	Replay a bearer token captured before logout	Token hash revoked server-side at logout; subsequent requests return 401	api_server.py /api/logout; authenticate_security_session()
3	IDOR / role manipulation	Clinician requests another clinician's session/report by ID	Non-admin requests scoped to caller's own clinician_id via _scoped_clinician_id()	api_server.py require_roles(), _scoped_clinician_id()
4	SQL metacharacter injection	Inject `' OR '1'='1` into identifier/text fields	Value bound via parameterized `?` placeholder; treated as literal data, not SQL	database_manager.py, api_server.py (see Section 3.1)
5	Path traversal on export	Attempt to redirect export path outside the protected backup directory	validate_encrypted_backup() rejects paths outside BACKUP_DIR	security_manager.py validate_encrypted_backup()
6	Direct plaintext DB access	Open the SQLite file directly with a generic tool, bypassing the API	Credentials/tokens are hashed, not plaintext; clinical fields are not, however, column-encrypted (see R-06)	database_manager.py schema; Risk Assessment Report R-06
7	Report/export tampering	Modify an exported report after creation	SHA-256 digest recorded at creation; tampering changes the digest and fails verification	security_manager.py create_encrypted_backup()/validate_encrypted_backup()
8	Audit log deletion/alteration	Edit or delete an audit_log row to hide an action	Hash chain breaks; verify_audit_chain() flags the first invalid entry	database_manager.py log_audit()/verify_audit_chain(); SEC-04
9	Consent bypass	Call a capture endpoint without a prior consent record	Server returns 403 "Consent verification is required"; fail-closed	api_server.py consent checks; SEC-03
10	Interrupted database write	Terminate the process mid-write during a session save	SQLite's atomic commit semantics prevent partial-row corruption; last committed state persists	sqlite3 transactional behavior (language guarantee, not custom code)

These are test-plan entries with source-grounded expected outcomes, not executed "Passed" results against a live target. An authorized dynamic penetration test executing scenarios 1–10 (and any others an independent tester identifies) against a running deployment remains a required external deliverable before an institutional production claim can be made.

7. Findings Summary

Category	High	Medium	Low	Notes
SAST (bandit)	0	13 (reviewed, false positive)	15 (hardening opportunity)	0 exploitable High/Critical findings
SCA (npm audit) — runtime deps	0	0	0	All shipped runtime advisories resolved via npm audit fix
SCA (npm audit) — devDependencies	7	1	0	Build-toolchain only; not shipped to end users
Regression tests	0 failures (code)	1 platform-only failure (SEC-05)	—	9/10 pass in this environment; 10/10 expected on Windows target
Conceptual penetration test	—	—	—	10/10 scenarios have a source-grounded expected mitigation; not independently verified live

8. Remediation Status and Outstanding External Evidence

- Completed in this review: SAST scan and disposition, dependency audit and non-breaking remediation, full regression suite execution, intrusion-monitor implementation and test coverage, conceptual penetration-test walkthrough.
- Outstanding (external, requires a live environment or people, per the thesis Appendix “Final Project Submission Boundary”): authorized dynamic HTTP/penetration test against a running deployment, clean installation evidence on an independent target workstation, signed User Acceptance Test forms, an institutional restore drill, and Windows-platform confirmation of SEC-05.

9. Conclusion

PsyClick Secure's source code shows no High or Critical static-analysis findings, a fully remediated runtime dependency set, and a passing regression suite covering every mandatory CS0029 authentication, authorization, consent, audit, and backup control. The remaining gaps — at-rest encryption of the live database, MFA, and live dynamic/penetration testing — are documented as explicit, prioritized follow-ups in the companion Risk Assessment Report rather than glossed over, consistent with the academic-integrity requirement that no test result be reported as passed unless it was actually executed.

10. References

Microsoft. (2023). How to: Use the Data Protection API (DPAPI). Microsoft Learn. https://learn.microsoft.com/windows/win32/seccng/cng-dpapi

MITRE. (2023). CWE-79: Improper neutralization of input during web page generation (cross-site scripting). Common Weakness Enumeration. https://cwe.mitre.org/data/definitions/79.html

MITRE. (2023). CWE-89: Improper neutralization of special elements used in an SQL command (SQL injection). Common Weakness Enumeration. https://cwe.mitre.org/data/definitions/89.html

npm, Inc. (2024). npm-audit: Run a security audit. npm Documentation. https://docs.npmjs.com/cli/commands/npm-audit

OWASP Foundation. (2020). OWASP Web Security Testing Guide (Version 4.2). https://owasp.org/www-project-web-security-testing-guide/

OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

Percival, C. (2009). Stronger key derivation via sequential memory-hard functions. Proceedings of BSDCan 2009.

PyCQA. (2024). Bandit: A tool designed to find common security issues in Python code. https://bandit.readthedocs.io/

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). Technical guide to information security testing and assessment (NIST Special Publication 800-115). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-115

The Penetration Testing Execution Standard. (2014). PTES technical guidelines. http://www.pentest-standard.org/